

须同时考虑实用方面的各种限制，例如，系统用户的要求、可以使用的编译程序、可以得到的软件工具、工程规模、程序员的知识、软件可移植性要求和软件的应用领域等。

2. 程序设计风格

良好的程序设计风格可以提高程序的可理解性、可复用性、可扩展性和健壮性，不仅能明显减少维护或扩展的开销，而且有助于在新项目中移植和复用已有的程序代码。

与可理解性相关的良好程序设计风格包括有意义的标识符、详细的注解和程序的视觉组织、清晰规范的数据说明和简单明了的语句构造，以及有效、合理、交互化与可视化的 I/O 设计等。与可复用性相关的良好程序设计风格包括提高功能的内聚、减小功能的规模、保持功能的一致性、将接口与实现分开、尽量不使用全局变量和利用继承机制等。与可扩展性相关的良好程序设计风格包括封装实现策略、利用多态性机制、避免使用多分支语句和精心设计公有服务等。与健壮性相关的良好程序设计风格包括预防用户的错误操作、检查参数的合法性、不要预先确定限制条件和先测试后优化等。

14.1.3 编码规范

应用编码规范对于软件本身和软件开发人员而言尤为重要，有以下几个原因：

(1) 好的编码规范可以尽可能地减少一个软件的维护成本，并且几乎没有任何一个软件在其整个生命周期中均由最初的开发人员来维护。

(2) 好的编码规范可以改善软件的可读性，可以让开发人员尽快并彻底理解新的代码。

(3) 好的编码规范可以最大限度地提高团队开发的合作效率。

(4) 长期的规范性编码还可以让开发人员养成好的编码习惯，甚至锻炼出更加严谨的思维方式。

编码规范一般会从命名规范、注释原则、编程风格规则和版本管理规则这四个方面来制定对应的要求。

常见的编码规范有命名规范、格式化规范、注释规范、函数长度规范、错误处理规范、冗余代码规范和通用开发模式规范等。

14.1.4 代码生成

在实际软件开发过程中，关注的重点是软件业务功能的开发及怎样保证业务流程的正确性。对于重复性的代码编写会占据大量的时间和精力，比如用户认证、权限管理等模块中的结构，用户、角色、权限等模块都有类似的结构。这部分代码往往有固定的规律，可以使用代码生成器来自动生成，减少手工的重复劳动，同时能把更多的精力用在关注更核心的业务逻辑以及软件的优化上面。

代码生成的实现步骤如下：

(1) 准备模板：生成文件的模板文件。

(2) 准备数据：生成文件所需要的关键数据。

(3) 定义合成机制：使用数据置换模板中的占位符，即生成新的文件的机制。